Practice 4 - Ex1

Learning Objectives

By completing this exercise, you will be able to:

- Implement all 5 RESTful CRUD operations (Create, Read All, Read One, Update, Delete)
- Use appropriate HTTP methods and status codes
- Create custom middleware (request logging)
- Handle errors with proper HTTP responses (400, 404)

Tasks

Complete the following tasks **in order**. Each task builds on the previous one.

Task 1 — Project Setup

- Import Express and create an application instance
- Add the JSON body parser middleware (`express.json()`)
- Define a `PORT` constant (use `3000`)
- Start the server with `app.listen()` and log a startup message

> **Verify**: Run `node server.js` — you should see your startup message in the terminal.

Task 2 — Logger Middleware

- Create a middleware function using `app.use()` that runs before every request
- It should log: the current timestamp, HTTP method, and request URL
- Format: `[2025-05-28T06:00:00.000Z] GET /users`
- Don't forget to call `next()` to pass control to the next handler

> **Verify**: After adding routes (next tasks), every request should print a log line in your terminal.

Task 3 — GET All Users

- Create a `GET /users` route that returns the full array of users as JSON

Task 4 — GET User by ID

- Create a `GET /users/:id` route
- Parse the `id` parameter from the URL
- Find the matching user in the array
- If found, return the user as JSON
- If not found, return status `404` with an error message

Task 5 — CREATE a New User

- Create a `POST /users` route
- Extract `name` and `email` from the request body
- Validate that both fields are provided — if not, return status `400` with an error message
- Generate a new `id` for the user
- Add the new user to the array
- Return the created user with status `201`

Task 6 — UPDATE a User

- Create a `PUT /users/:id` route
- Find the user by ID
- If not found, return status `404`
- Update only the fields that are provided in the request body (`name`, `email`)
- Return the updated user

Task 7 — DELETE a User

- Create a `DELETE /users/:id` route
- Find the user index by ID

- If not found, return status `404`
- Remove the user from the array
- Return status `204` (No Content) with an empty response

API Endpoint Reference

Use this table to verify you've implemented all required endpoints:

Method	Endpoint	Description	Success Status	Error Status
`GET`	`/users`	List all users	`200`	—
`GET`	`/users/:id`	Get a single user	`200`	`404` Not Found
`POST`	`/users`	Create a new user	`201`	`400` Bad Request
`PUT`	`/users/:id`	Update a user	`200`	`404` Not Found
`DELETE`	`/users/:id`	Delete a user	`204`	`404` Not Found

Testing Your API

Start your server:

```
node server.js
```

Using curl

```
# GET all users
curl <http://localhost:3000/users>

# GET user by ID
curl <http://localhost:3000/users/1>

# CREATE a user
curl -X POST <http://localhost:3000/users> \\
  -H "Content-Type: application/json" \\
  -d '{"name": "Test User", "email": "test@example.com"}'
```

```
# UPDATE a user
curl -X PUT <http://localhost:3000/users/1> \\
  -H "Content-Type: application/json" \\
  -d '{"name": "Updated Name"}'

# DELETE a user
curl -X DELETE <http://localhost:3000/users/1>
```

Using Thunder Client / Postman

1. Open Thunder Client (or Postman)
2. Create requests matching the endpoint table above
3. For POST and PUT requests, set the body to JSON format
4. Verify the response status codes match the expected values

---

Submission

Submit your completed work using **one** of the following methods:

- **GitHub**: Push your code to a repository and share the link
- **Zip**: Compress the `EX-1/` folder (exclude `node_modules/`) and submit the `.zip` file

---

Good luck! 🚀